

CHANGE POINT DETECTION



Mentor: Ravi Kothari

Intern: Amod Agrawal

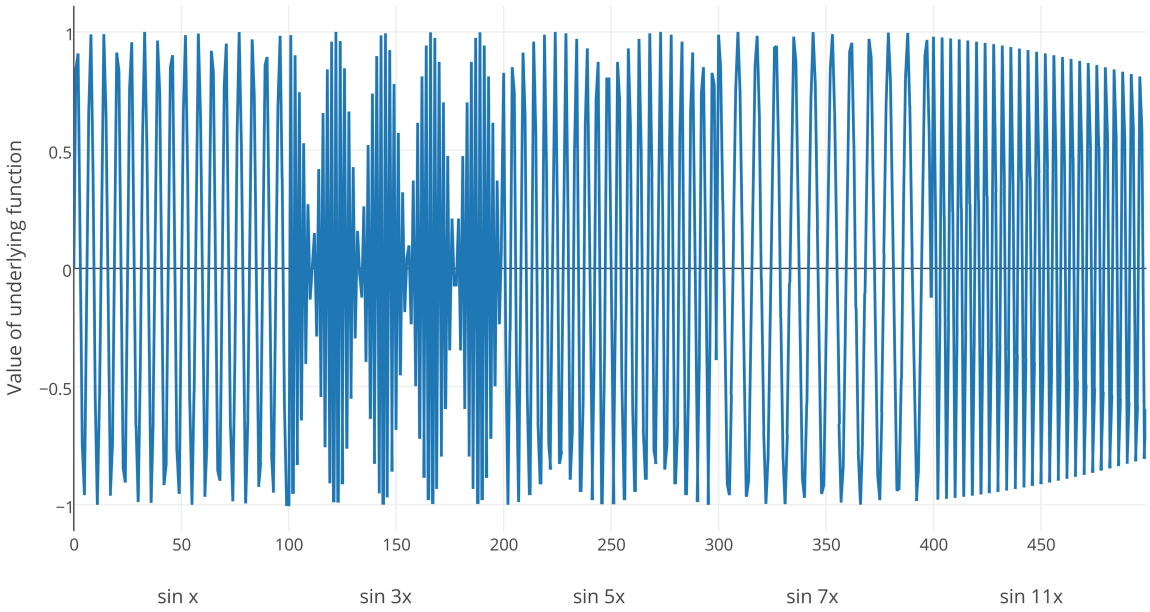
Motivation for Change Point Detection using Edit Distance

- Normal system behavior can be characterized in terms of the nominal behavior of a collection of (business and IT) variables
- Faults often result in the deviation in the nominal behavior of one or more of the variables
- Semantic errors go undetected; their effect often is meaningless numbers in reports that get produced (often after a delay of several days from the point of first occurrence of the error).
- Change point detection identifies deviations in the nominal behavior of the identified variables. However, classical change point detection has typically worked only with numerical data
- Change point detection typically works only with numerical data. By basing our method on Edit Distance, we ensure applicability in numerical and non-numerical domains.

Quantization of Numeric Data

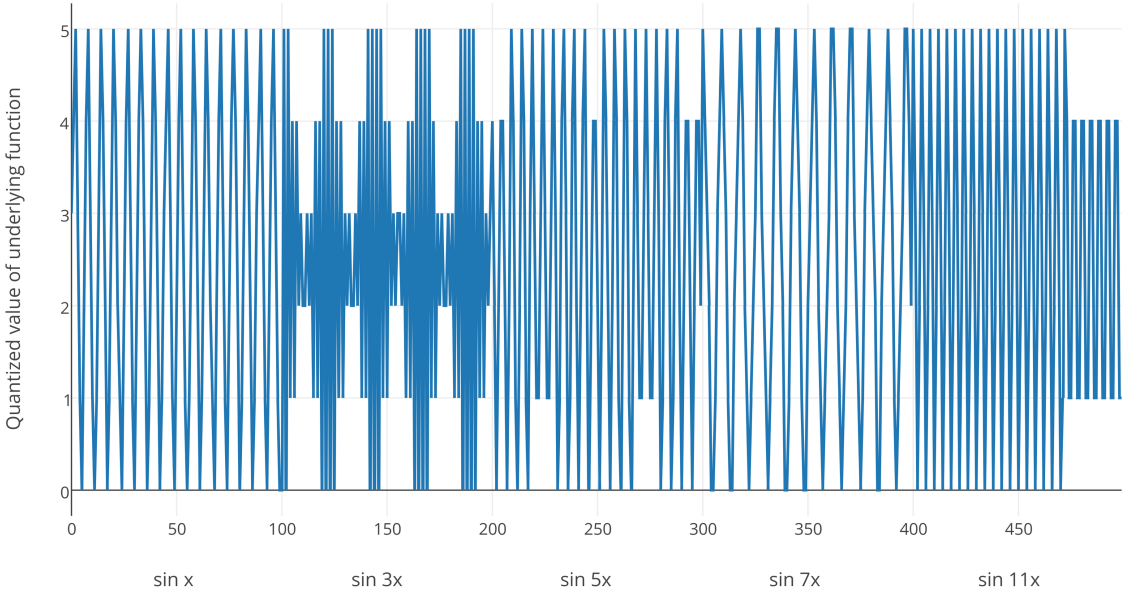
- The entropy at the output of a quantizer is equal to the average mutual information between un-quantized and quantized random variables.
- The numeric data is quantized into a fixed number of levels (six in this case) using maximum entropy quantization.

Raw Data with underlying function changing at: 100, 200, 300, 400



- For output entropy of the quantized data to be maximum, probability that a data point lies in each level of quantization is equal.
- Therefore, the data is quantized to different levels, such that the number of data points in each level are equal.
- Probability that a data point lies in any of quantized levels is $1/N$. (N is the number of quantized levels)

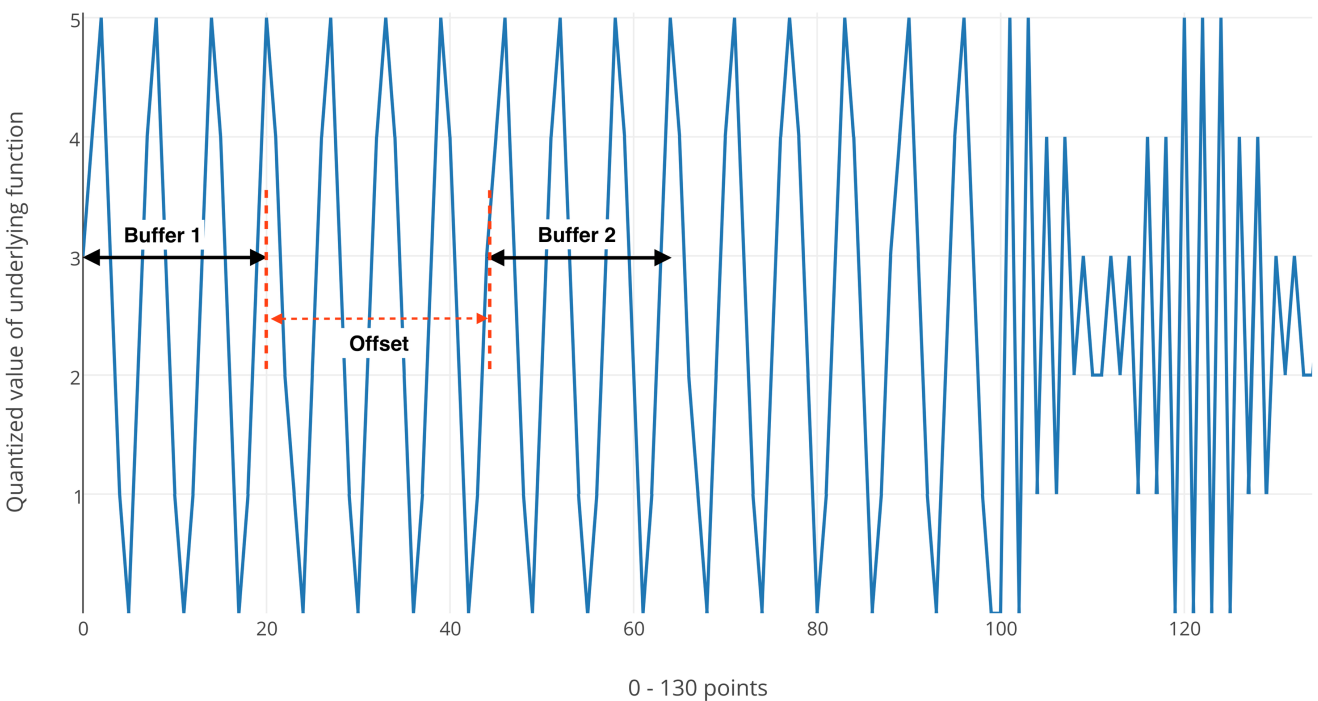
Maximum Output Entropy (MOE) Quantized Data on 6 levels with underlying function changing at: 100, 200, 300, 400



Algorithm for Change Point Detection

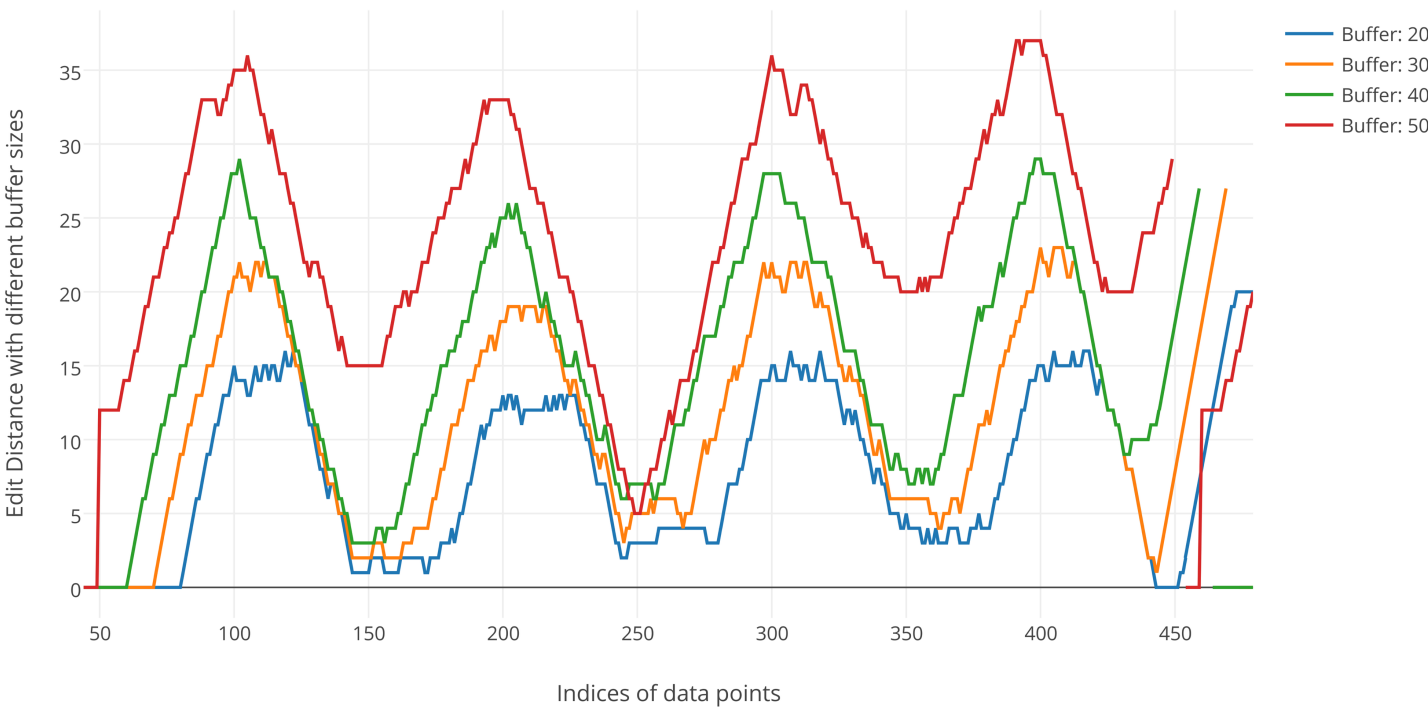
- Our algorithm for Change Point Detection uses Edit Distance (a.k.a Levenshtein's Distance).
- Given a string A and a string B, Edit Distance $d(A,B)$ is the minimum number of operations required to convert string A to B.
- Edit operations are defined as:
 - Insertion of a symbol ($\epsilon \rightarrow x$) with a weight of 1
 - Deletion of a symbol ($x \rightarrow \epsilon$) with a weight of 1
 - Substitution of a symbol ($x \rightarrow y$) with a weight of 1
- We consider two buffers of same size S:
 - First Buffer (B_1) contains first S elements of quantized data (Indices: 1 to S)
 - Second Buffer (B_2) contains next S elements of quantized data (Indices: S+1 to 2S)
- We move B_2 through out the entire data to find an offset with minimum Edit Distance w.r.t. B_1 .
 - This is done to detect periodicity in the data.

Finding offset with Minimum Edit Distance



- After calculation of the offset, we move into the second part of the algorithm:
 - Both the buffers, B_1 and B_2 move through the data while maintaining a difference of calculated offset value between their indices.
 - In each iteration, B_1 and B_2 lose their first element and the next respective element in the quantized data is added to them.
 - B_1 and B_2 are compared using Edit Distance Algorithm in every iteration and the value of Edit Distance is plotted.

Change Points Detected with maximum Edit Distance at: 100, 200, 300, 400



Inference from the results of the algorithm

- The point at which the value of Edit Distance increases and reaches a local maxima, is found to be a Change Point.
- Different plots of various buffer sizes (such as 20, 30, 40, 50) verifies that all buffer sizes reach a local maxima at a point of change.
- All plots (with different buffer sizes) follow a similar construct.
- Magnitude of change doesn't matter. It's the change in the underlying function being followed by the data that is detected.
- This algorithm works not only for numeric data, but also textual data.
- This algorithm has been tested on many datasets:
 - Yearly Periodic Weather data
 - Eye Movement data
 - IBM Stock closing value
 - Sunspots data

Applications

- Syntactical errors made by a programmer are caught by the compiler, however, semantical errors are caught after the execution of the program and analysis of the results.
 - Using Change Point Detection, we can detect these errors during the execution of the program. If the value of any variable goes out of it's nominal range, we raise a red flag.
- Large corporations are always monitoring data centers and all the results are logged every hour. These logs are usually analyzed at the end of the week, and if something went wrong: a team has to backtrace the error to the point when it was created in the system.
 - Using Change Point Detection, we can detect these points of fault in real time.
- Change Point Detection algorithm can also used to monitor a large amount of network traffic and usage of the links. Unusual amount of traffic through links can be used to detect faults in a Network.

Future Work

- We plan to improve the algorithm by segmentation of the data based on some statistical approaches.
- Instead of grouping the data in segments of equal size (buffer size), we plan to segment the data in segments of non uniform sizes, such that variance of these segments is minimum.

References

- [1] Messerschmitt, D. "Quantizing for maximum output entropy" ISSN :0018-9448 DOI: 10.1109/TIT.1971.1054681 IEEE Publication
- [2] R. Wagner; M. Fischer (1974). "The string-to-string correction problem". J. ACM 21: 168–178. doi:10.1145/321796.321811
- [3] Miodrag Lovrić, PhD Marina Milanović, MSc Milan Stamenković "ALGORITHMIC METHODS FOR SEGMENTATION OF TIME SERIES" UDC 004.65.021:519.246.8